

2026 CSP-S 复赛赛前模拟赛 Day2 复盘

比赛：代码源新 OJ · 2026 CSP-S 复赛赛前模拟赛 day2 · OI 赛制 · 灵活时间 4 小时（你是北京时间 10:10 到 14:10） · 四题都是 1 秒 / 1024 MiB · 必须 freopen

结果：130 / 400 (A 100, B 0 未交, C 30, D 0)。比 Day1 的 45 分多 85 分

分析依据：归档（题面 + 题目列表页快照，里面有你全部 6 次提交）、赛时录音转写（10:03 到 14:10）、虚拟机赛后桌面和 Downloads（每个代码版本、大样例、D 的 checker）。四题正解我都写了，用大样例和暴力对拍验证过。草稿纸按你说的没要。

动画讲义：动画讲义/Day2四题动画复盘.html（在线版：

<https://claude.ai/artifact/MdcsTtyD2D6kniFuBdst8s>），B 题“不用重算每个 f”、A 题的反例 11011、D 题暴力的 bug 都有逐帧动画。

第二版（09-28）按你的反馈改了四处：① B 你卡住的不是“字典树退化链”，是“每个 i 都要把 $j = 1..i$ 全算一遍”（录音 11:49、12:45 原话）；② A 你 10:59 说的记忆化和我以为的数位 DP 不是一回事；③ 11:22 和 12:31 两次 Code::Blocks 报错不怪你，是文件夹名 `problem_185(1)` 里的括号（第十一节）；④ 时间线按分钟重写，加了“暴力为什么来晚了”（第六节）和“我是怎么做出来的”（附四）。

零、一页结论

先说好消息：A 是你自己推出来的。连续 1 算两项、中间只能夹一个 0——这些全是你赛时推出的结论。最后 18 分钟你写了暴力、开了对拍，2 分钟就抓到反例，13:35 满分。这正是 Day1 没有的能力，要保住。

但这场还是丢了很多分，原因就三条：

- 暴力来得太晚，不是因为不会写，是没认出来它就是你要的东西。你自己总结的三条原因，录音里都有证据（第六节）：
 - 10:16 你说“10 的 6 次方……而不是进行暴力”，一开始就把暴力排除了
 - 11:16 你在 paper.txt 写下 $a + b - (0 + b) = a$ 。这就是暴力：枚举 b，数两边 1 的个数。你当时把它当成“题目换个说法”
 - 12:13 你说“找一个很强的样例，关键我自己也算不出来答案”。这时最需要的就是暴力
 - 直到 13:17 “等一下，有可能是你生成两个”，暴力才写出来，对拍 2 分钟就找到反例。A 一共花了 2 小时 37 分，占全场 65%
- B 你离正解只差一步。11:45–11:49 你已经想到“每层最大值 + 插入时路径上全加一”，这是正解的前一半。卡住你的是 11:49 这句：“现在我能高速算出来每一个，但这复杂度差得远”——每个 i 都要把 $j = 1..i$ 全算一遍， $O(n^2)$ 。缺的一步是：插入一个串，f 数组里只有很少几格会变，只改这几格，和跟着改（四.B 有逐步表格）。B 的 10 分暴力也没写。
- D 两次提交都是 0 分，而且都是能避免的。

- 14:01 那版暴力：枚举狼的分布时，你按 `i` 的二进制位数取位，结果只试了“最后一个人是狼”的那一半情况。题面样例三组答案恰好都是“最后一个是狼”或无解，所以样例过了；大样例 `sample2`（正好是子任务 1 的 1000 组数据）一跑就会错 277 组，你没跑。
- 14:09 那版：`solve2` 先用后声明，编译错误。离结束还有 14 秒，交之前没编译（可执行文件是 14:00 的，源码是 14:09 的）。

按你现在的水平，这场合理分数：

题	实际	稳拿线	努力线	差在哪
A	100	100	100	拿到了，但花了 65% 的时间
B	0	10	40	10 分暴力没写；11:49 的想法写出来就接近正解
C	30	30	30	拿到了。60 分以上要二维偏序，现阶段不强求
D	0	10	25	暴力少枚举一半；全是 L 的 15 分是两状态 DP
合计	130	150	195	

下场只改三件事：

1. 把暴力当成找反例的工具，不只是骗分：出现下面任一信号，5 分钟内写暴力 + 小数据对拍——大样例不对但你觉得自己考虑全了；自己算不出小数据的答案；纸上写出了“答案 = 在所有 X 里取最好”的式子。
2. 交之前三件事：刚编译过、样例和大样例全过、交的就是测的那个文件。做不到就不交。
3. T+1:00 还没交过任何题，立刻停下写暴力交上。这场第一次提交是 T+2:35。

一、证据完整性检查

已看到：

- 归档：4 题题面、时空限制、子任务表；题目列表页快照里有你每题最终得分和 6 次提交的时间、状态、耗时
- 录音转写 1 份（北京 10:03 到 14:10，全程）
- 虚拟机赛后桌面 + Downloads：A 的 7 个版本、C 的 2 个版本、D 的 1 个版本、B 的开头；三题大样例；D 的 checker；对拍文件夹

没看到：

- 榜单和名次（OI 赛制，平台没给看）。所以本报告没有同分段对比
- 其他选手的代码
- 草稿纸（你说不用）；回收站（这次导出里没有 Day2 的回收站）

我纠正的数据：

项目	归档里写的	实际	怎么查出来的
本人提交	0 次，“完全未提交 4 题”	6 次，130 分	提交接口返回 403，但 <code>01_比赛信息/题目列表服务器页面.html</code> 里有提交表

项目	归档里写的	实际	怎么查出来的
D 的题名	“0 Compile Error”		归档脚本把状态栏当成了题名。“0 Compile Error”其实是你 D 题的最后状态
文件时间	—	桌面导出 ZIP 里的时间 +8 小时是北京时间	用 6 次提交时间逐一对齐（例：T4.cpp 06:09 ↔ D 提交 14:09:46）。不再是 Day1 的 +12
录音起点	转写文件头写 2026-09-26 22:03	北京 10:03:02	文件头是纽约时间；用 3 次提交对齐：录音 2:42:11 说“交上去了”↔ 12:45:13，3:03:33 ↔ 13:06:36，4:06:44 ↔ 14:09:46

下文时间都是北京时间；“T+2:35”指开赛后 2 小时 35 分（开赛 10:10）。

二、比赛总画像

题	名字	本质	你的状态	分
A	乘法	用最少几个 $\pm 2^k$ 凑出 a （有符号二进制），答案 = $2 \times \text{项数} - 1$	3 次提交：RE → WA → AC	100
B		字典树 + 每层最大计数，增量维护答案	想了 30 分钟，没交	0
C		前缀差 + 二维偏序 DP； $n = 2 \times 10^6$ 要利用“每一步两维同涨同跌”	$O(n^2)$ DP，TLE	30
D		村民链 + 总狼数只在“左看/右看交接处”被钉死	暴力 0 分，最后一交 CE	0

6 次提交（全部来自页面快照）：

时间	题	结果	交的是哪份
12:45:13	A	0 Runtime Error	problems/T1.cpp（12:22），while 条件写成了 <code> </code> ，数组越界
13:06:36	A	0 Wrong Answer	time.cpp（13:02），大样例输出 729，你当时已经知道应该是 671
13:35:19	A	100 Accepted	duipai_T1/wa.cpp（13:33），加了 check 函数
13:51:49	C	30 Time Exceeded	debug/main.cpp（13:49）， $O(n^2)$
14:01:08	D	0 Time Exceeded	14:00 编译的暴力（枚举漏一半）
14:09:46	D	0 Compile Error	problems/T4.cpp（14:09），solve2 未声明

时间都花在哪了：

去向	时长	占比
A（10:10–11:39、12:13–12:45、12:59–13:35）	2 小时 37 分	65%
B（11:39–11:52、12:45–12:59、14:01–14:04）	约 30 分钟	12%
C（11:52–12:05、13:35–13:51）	约 29 分钟	12%
D（12:05–12:13、13:51–14:01、14:04–14:09）	约 23 分钟	10%

其中环境问题约 12 分钟：复制粘贴失灵（10:44、12:38）、终端报 `word unexpected`（11:22–11:29，7 分钟）、Code::Blocks “抽风”后改用终端（12:31）。比 Day1 的 30 分钟少了一大半，“5 分钟修不好就回终端”这条规则起作用了。

一句话判断：想法质量在变好（A 全靠自己推，B 想到了一半，C 想到了前缀差），但“想法 → 分数”这一段还是漏：暴力写得太晚、交的东西没测、四题读完用了近 2 小时。

三、时间线（按分钟）

对齐方法：录音 0:00 = 北京 10:03:02（用 3 次提交时间反推：录音 2:42:11 “交上去了”↔ 12:45:13；3:03:33 ↔ 13:06:36；4:06:44 ↔ 14:09:46）；桌面导出里的文件时间 +8 小时 = 北京时间。引号里是录音原话（语音转写，个别同音字按上下文改了）。加粗行是提交，评价里 ✓✓ 是关键一步，✗ 是丢分的地方。

北京时间	T+	发生了什么（引号里是你的原话）	评价
赛前			
10:03		开录音、开虚拟机、拿草稿纸、登录	
10:06		调分辨率（60Hz），打开比赛页面	
10:07:52		“这一次要进行文件输入输出”	✓ 注意到 freopen
10:08		打开 Code::Blocks，“确认未打开任何项目”；放大代码；打开 Sublime Text 做记录	✓ Day1 的工程陷阱规则做到了
10:09:05		参加比赛，看到四个 emoji 题名，开读 A	
A 第一段 (10:10–11:39, 89 分钟)			
10:10:37	0:00	“先把所有的附件下载下来”	✓ problem_185.zip 10:11
10:11–10:15	0:01	读懂题意： $x \times a$ 拆成若干个 $\pm(x \text{ 左移 } k \text{ 位})$ ；用样例 1 验证 $16 - 1 = 15$	✓
10:15:37	0:05	“只要减少项数就行了……答案一定是 N 项”，paper.txt 写下 $2^n - 1$, n is xiang shu	✓ 代价公式对
10:16:19	0:06	“10 的 6 次方，说明是有通用的方法的，而不是进行暴力。要么 DFS 加记忆化，要么有个好方法”	✗ 一开始就把暴力排除了。子任务 1 ($ a \leq 10$, 20 分) 就是暴力
10:17–10:18	0:07	方法一：直接按 1 拆（项数 = 1 的个数）；方法二：“位数前面加 1，减去某个数”	只考虑了“全加”和“最高位减一次”两种
10:18:02	0:08	“第二步是下载第二遍，防止附件有用”	△ 产生了 problem_185(1)，后面两次 Code::Blocks 报错都出在它身上
10:19–10:24		样例 2 “不是人看的”，自己编 101110110，用计算器算两种方法：6 项 vs 3 项	✓ 手玩小例子
10:25:04	0:15	定计划：“第一步 popcount，第二步 大数减它再 popcount，第三步取 min”	没验证这两种是不是全部情况
10:25–10:33		写代码：10 ⁶ 位，手写二进制高精度减法	
10:33–10:40		进 debug 工程下断点，调借位	
10:43:15	0:33	“可以把项目关了”	✓ 调完关工程
10:43–10:46		复制粘贴失灵，用 Sublime Text 中转	环境问题，约 3 分钟

北京时间	T+	发生了什么（引号里是你的原话）	评价
10:46–10:47		终端编译，跑样例 2，文件名打错（下划线）	
10:48:27	0:38	样例 2 输出 947，答案 671：“怎么错了？”	止损信号 1。Downloads/ans.out 10:48 = 947
10:49:05	0:39	“我发现个事情，为什么一定要在最高位呢？”	✓ 找到病根：减的那一项不一定在最高位
10:51–10:56		“我先搞成某个数，再给它减掉……两部 cost……往下递归”	方向对
10:58:17	0:48	“这一题能否用分治来写？记忆化？”	
10:59:22	0:49	“如果跑个记忆化，记录所有出现过的每一位……总共的位置太多了。那算了”	你想记的是“哪些位用过”这种整体状态，确实太多。能行的状态是“做到第几位 + 进位 0 或 1”，只有 $2n$ 个
11:00–11:02		“应该有个贪心的策略”“分成若干段各自求再加起来……复杂度是指数的”	
11:05:04	0:55	“答案满足单调性吗？二分？……验证也比较难”	
11:07:11	0:57	“我只会用一次加法和一次减法……两个位置都是一，还不如进位之后只用一个一……所以最终只要把它拆成 A 减 B”	✓✓ 关键转化： $a = P - N$
11:10:33	1:00	“加和减放到一起算……最小化加的位数和减的位数之和”	✓ 目标写对了。T+1:00，还没交过任何题
11:12:12		“某一段能变小就直接变小，后面继续往下贪……会不会后面再移一个赚得更多？”	贪心不敢定，也没法验证
11:16:31	1:06	“A 加上 B 减去 (0 加上 B) 还是等于 A……要使它加上 B 以后一的个数最少”，paper.txt 写下 $a + b - (0 + b) = a$	✓✓ 这就是暴力：枚举 B，数 A+B 和 B 的 1。你当时把它当成“题目换个说法”
11:19:21		“卡着的原因：两个是连锁的，一个动了，另一个也会动”	
11:20:17	1:10	“再想想第二题吧……第一题有这么难吗？”转念先数样例 2 里 1 的个数	✓ 想打表
11:22–11:29		Code::Blocks 运行报 Syntax error: word unexpected；“别的代码有这样的吗？没有”“注释掉一些还不对”“不会是编译器的问题吧”	7 分钟。根因：文件夹名 problem_185(1) 里的括号
11:30	1:20	写 tot.cpp 在终端跑：样例 2 有 474 个 1，答案 671	1.out 11:30 = 474
11:30–11:32		“474 跟 671 好像没有什么关系”“有没有可能跟 lowbit 有关？”“这一题能不能被扔了？”	
11:33:06	1:23	“它一定是长这个样子的：几个零之间肯定只隔一个一”	✓ 这就是答案的结构（非相邻形式）
11:33:58		“再想五分钟……依旧没有做过类似的题”	
11:35:14		“不如还是顺着我原来的思路往下研究”	
11:38:59	1:29	“没有找到任何破局点。算了，转到下一题”	A 第一段结束

北京时间	T+	发生了什么（引号里是你的原话）	评价
B (11:39–11:52, 13 分钟)			
11:39–11:41	1:29	读 B：“最长公共前缀，这不是字典树吗”	✓ 秒认模型
11:41–11:43		画字典树：“选出来的串深度要尽可能深，在同一个子树上”“二分当前深度”	✓
11:43:34		“这一题甚至能不能把每一个给算出来？不能以 N 方算出来”	已经看到瓶颈：每个 i 都要算 i 项
11:45:18	1:35	“我给每一层求个最大值”	✓✓ 就是正解里的 $mx[d]$
11:47:02		“实际上都不用二分……从头到尾把整个字典树建出来，边更新边把这个值存下来”	✓✓
11:48:55	1:38	“插入一个串，这条路径上全加一，如果这是最大值，上面的每一层都要取一遍最大值”	✓✓ 正解的前一半
11:49:47		“字典树如果退化成链……难办了。现在我能高速算出来每一个，但这复杂度差得远了去了”	真正的卡点：mx 会维护了，但每个 i 还要把 $j = 1..i$ 全算一遍， $O(n^2)$
11:51:55	1:41	转 C	
C (11:52–12:07, 15 分钟)			
11:52–11:54		读 C：“输入相当于二进制”“复杂度非常大，那只能 DP”	
11:54:38		“记录上一次切割点……最后一次切割点”	
11:56–11:58	1:48	$f[i]$ ：好段 $f[i] = f[j-1] + (i-j+1)$ ，否则 $f[i] = f[j-1]$ ；“三个前缀和……这是 N 方的，不满足”	✓ 30 分的做法完整了
11:58:39		“慢就慢在我们不能直接知道前面这个的值”	
12:00–12:02		“A 大于 B 加 b……只要知道 A-B 的值”“A-B 要大于 b，A-C 要大于 c”	✓ 前缀差，就是二维偏序
12:03:52	1:54	“如果我以……I 减去 F1，最终只要做一个简单处理”	✓ 正解里的关键量就是 $i - f[i]$
12:04:58		“压根就不用做前缀和，往后倒退的时候一点点累加就行了”	✓ 你的代码就是这么写的
12:05:31	1:55	“没有啥好的思路……最后再来骗，N 方的分非常好骗”“30 分手到擒来”	✗ 想到了没写，推迟了 1.5 小时
12:06:56		“继续看第二题，第二题 30 分应该比较好骗”	
D (12:07–12:13, 6 分钟)			
12:07:06	1:57	读 D：“第四题我估计更难”	T+1:57 才读完四题
12:08:39		“我这边咋还有个 checker？”读完：“他好像只是验证，没有给出答案”	✓ 执行卡“先看 checker”做到了

北京时间	T+	发生了什么（引号里是你的原话）	评价
12:12:16		“假设所有人说的都是真话……一旦不单调……特殊情况太多了”	开始分类讨论
A 第二段 (12:13–12:45, 32 分钟)			
12:13:27	2:03	回 A：“我好像发现了一些规律……我找一个很强的样例，关键我自己也算不出来这一道题答案应该是多少”	← 这一刻最需要暴力：暴力就是“算得出答案”的程序
12:16:53		“假设最后答案一定是个 00……1 的”	
12:18:02	2:08	“OK，我会了，第一题我会了，我发现了诀窍”	连续的 1 两项、单独的 1 一项。 paper.txt 里那个 22 位的例子算到 13
12:18–12:22		写 v2 (problems/T1.cpp 12:22) : while (point <= n a[point] != 1)	✗ 应为 &&，越界
12:25:15		“重新解压一个”（桌面上又一个 problem_185(1)）	又一个带括号的文件夹
12:26:54		终端“编译成功”；12:27:29 “第一次让你对了”（样例 1）	
12:28:05	2:18	样例 2：“671……729”	止损信号 2
12:30:58		“Code::Blocks 又抽风了，算了，还是用终端运行”	和 11:22 同一个原因： problem_185(1)
12:33		改系数 (2 / 4 / 4) → 853 (桌面 problem_185(1)/T1.cpp 12:33)	✗ 碰运气
12:38:15		“Linux 复制粘贴……你复制的东西被关掉的时候，那你就行了”	✓ 自己发现的规律，是对的
12:39–12:41		“614 改完了之后……594……709”	✗ 继续试系数
12:42–12:43		“我这个算法就是这个样子，为什么不对呢？”“我在 debug 当中那个代码不对”	发现自己跑的和改的不是同一份
12:45:13	2:35	第一次提交 A → RE。“把我认为最正确的思路交上去了”（你说：先交一下再说，先把代码保存）	交上去的是 12:22 那份越界版
想 B、算分、想 D (12:45–12:59, 14 分钟)			
12:45:27		“第二题……你不挨个算，你怎么能做到的呢？”	B 的真正卡点，原话
12:46–12:50		算分：“第二题顶多拿 40……第三题 30 分稳了……第四题暴力 10 分，第 2 个子任务 15 分，这样子才 80”	✓ 在算分
12:51–12:57		D 子任务 2：“第一个人肯定说 0，不是 0 直接拖出去……比它增加 1，左边那个人就是狼……列出所有情况，总共 8 种……还是太复杂了”	分类讨论爆炸

北京时间	T+	发生了什么（引号里是你的原话）	评价
12:59:24	2:49	“继续保第一题……第二题拿个 10 分应该比较容易”“最后一题那个做法比正解还复杂，不值得”	
A 第三段 (12:59–13:35, 36 分钟)			
13:02:04	2:52	手测小数据：10、110、1110，都对	手编的数据太“整齐”，碰不到反例
13:05:18		“突然想起来……没有注释掉，我重新交一遍，干干净净”	
13:06:36	2:56	第二次提交 A → WA (time.cpp 13:02, 样例 2 输出 729)	
13:07:16	2:57	“好像不是代码问题，是思路有问题。但我认为我的思路没有问题”	✓ 关键转变（你自己说的）
13:08:35		“第一题我再下载一遍看看” (problem_185(2).zip 13:09)	
13:10:11	3:00	“主要是我自己找反例找出来，我也没有啥暴力代码，我有吗？我会写暴力吗？我最近暴力咋写？”	正好是你说的三条原因
13:10–13:17		“随机生成一个序列，验证它是否是……漫无目的地随机生成”；13:17:16 “算了……放弃了”	
13:17:45	3:07	“等一下，有可能是你生成两个”	✓✓ 11:16 那个式子回来了：枚举 b
13:18:22		“我又想到第二种构造方法……把第一种全删了”	
13:20:50		暴力 ac.cpp (13:22)、gen (13:23)、对拍脚本写好	
13:21:40		“1000 组没有任何问题”→“突然想起来，我一直一个数据都没生成呢”	gen 还是空模板
13:22:49	3:12	“OK，出错了”	✓
13:24:06		手算：“2 的 9 次方减去 2 的 6 次方，再加 2 的 1 次方……我代码输出 7，我明白哪里出问题了”	✓ 反例到手
13:28:14		“中间有个 0，你付出额外代价，但是你会少一次……最多接受中间只有一个 0”	✓ 最后一块拼图
13:31:08		“要不然我把它封装成一个（函数）”→ check()	
13:32:44		“4 加等于 2，OK 我明白了，继续拍”	
13:33:34		“好像没问题了，第一题已被攻破”；13:34:14 “671 OK”	
13:35:19	3:25	A → 100 (duipai_T1/wa.cpp 13:33)	暴力写好到满分：18 分钟
C 拿分 (13:35–13:52, 16 分钟)			
13:35–13:40		写 C (写进了 T2.cpp, 13:40)，断点看解码	
13:46:45		“我找到了，我哪里出错，干了件非常蠢的事情”	

北京时间	T+	发生了什么（引号里是你的原话）	评价
13:48:51		“大于等于我搞错了”	✓ 样例 1 抓住了读题错
13:49		定稿 (debug/main.cpp 13:49) ; “文件名是啥来着? chess”	
13:50:54		“好像交错题了……不对, 没点上去”	
13:51:49	3:41	C → 30	✓
13:51:51		“第一题满分……第三题做了防护, chess.in / chess.out, 应该 30 分……130 到手”	✓ 算分
D 暴力 (13:52–14:01, 9 分钟)			
13:52–14:00		下载 checker (chk.cpp 13:53) , “我用 checker 来写”, 搬 isValid 枚举	✓ 搬判定函数; ✗ 按 i 的二进制位数取位
13:59:49		“OK, 对了” (题面样例过了)	样例恰好测不出这个 bug
14:01:08	3:51	D → 0 (TLE)	
最后 9 分钟			
14:01:36		“那题写到 T2 去了, 那我就写到 T3 去”, B 写进 T3.cpp	
14:03:56		“第二题我有点难算……字典树难写”	
14:04:17	3:54	“第二题十分性价比不高, 来抢第四题”	✗ B 的暴力 10 分钟 10 分
14:04:36		“保证 d_i 为 L……全往左看, 我来继续骗”: 写 solve2	✗ solve2 写在 solve 下面, 没声明
14:09:46	3:59	D → CE (T4.cpp 14:09, 可执行文件还是 14:00 的)	没编译就交
14:10	4:00	结束	

四、逐题复盘

A 乘法 (100 分, 用时 2 小时 37 分)

题目本质：把 a 写成若干个 $\pm 2^k$ 的和，项数最少是多少（每项一次左移，项与项之间一次加减，所以答案 = $2 \times \text{项数} - 1$ ）。最少项数就是 a 的“非相邻形式” (NAF) 里非零位的个数。

你的思路复原（每一步都是你自己推的）：

- 10:15 答案 = $2 \times \text{项数} - 1$ ✓
- 10:16 “10 的 6 次方, 说明是有通用的方法的, 而不是进行暴力” ✗
- 10:17 两种构造：按 1 拆 / 高一位减补数, 取小 → 10:48 大样例 $947 \neq 671$
- 10:49 “为什么一定要在最高位呢?” ✓ 找到病根
- 11:07 $a = P - N$: 加的项合起来是 P , 减的项合起来是 N , 项数 = P 的 1 的个数 + N 的 1 的个数 ✓✓
- 11:16 写下 $a + b - (0 + b) = a$, “要使它加上 B 以后一的个数最少” ✓✓ 这就是暴力
- 11:33 “几个零之间肯定只隔一个一” ✓

- 8. 12:18 连续一段 1 用“高一位减最低位”两项搞定，单独的 1 一项 ✓ (v4，大样例 729)
- 9. 13:17 “有可能是你生成两个” → 暴力；13:22 对拍出错；13:24 手算反例
- 10. 13:28 一段 1 中间夹一个 0 时，多花一项就能把后面接上；夹两个 0 就断开 ✓ (v5，AC)

你的 13:06 版本 (v4) 为什么错——最小反例 **11011** (= 27)：

- v4：11 一段两项，11 又一段两项，共 4 项，输出 7
- 实际：27 = 32 - 4 - 1，3 项，答案 5。中间那个单独的 0 让两段可以合并

我把 12 位以内的所有 a (4095 个) 都和暴力对过：v4 错 1085 个，v5 (你最后交的) 一个不错。

你偏掉的三个地方：

时刻	你说的	问题	应该怎样
10:16	“10 的 6 次方，说明有通用的方法，而不是进行暴力”	只把暴力当骗分，看到 10^6 就不写了	暴力有三个用处：拿子任务分、给你小反例、打表找规律。 $ a \leq 10$ 的 20 分就是暴力
10:59	“跑个记忆化，记录所有出现过的每一位……位置太多了，算了”	你想记的是“哪些位用过”这种整体状态，确实太多，这个判断没错	换一个状态就行：“做到第几位 + 往上进位 0 还是 1”，一共 $2n$ 个 (下面的数位 DP)
11:16 → 13:17	写下 $a + b - (0 + b) = a$	没认出这个式子能直接跑	for b 从 0 到 4a，数 a+b 和 b 的 1，取最小。这就是 13:17 的暴力

不需要任何灵感的正解：数位 DP (和你 10:59 想的记忆化不同，它只记进位)

从最低位往高位做，每一位最后填 -1、0、+1 中的一个，状态只记“往上一位进位 0 还是 1”：

当前位 + 进位	可以怎么填	新进位	项数
0	填 0	0	不变
1	填 +1	0	+1
1	填 -1 (相当于借了一个 2)	1	+1
2	填 0	1	不变

最后如果还有进位，再加一项。15 行，见 正解代码/A_乘法.cpp。这是这类题的通用解法：二进制、加减、“最少用几项”，就从低位做，状态记进位。

如果 11:16 写了暴力会怎样：暴力 A_暴力.cpp 12 行 (你 13:17 写的那份几乎一样)。打出 1~64 的答案，“中间一个 0 可以吞、两个 0 断开”一眼就能看出来。按你 13:17 → 13:35 的速度，A 在 11:40 前后就能过，多出来近 2 小时给 B 和 D。

下次识别信号：

- 二进制 + 加减 + 最少项数 → 从低位做，状态 = 进位
- 纸上出现“A = X - Y”这种等式 → 枚举一边，就是暴力

专项：LeetCode 2571 《将整数减少到零需要的最少操作数》 (同一题，数据小)，用数位 DP 写一遍，再用贪心写一遍。

B (0 分, 未交)

题目本质：对每个 i , $f(i, j)$ = 前 i 个串里选 j 个的最长公共前缀，求 $\sum_j f(i, j) \text{ xor } j$ 。

你的思路复原：

- 11:40 “最长公共前缀，这不是字典树吗” ✓
- 11:41 “选出来的串深度要尽可能深，在同一个子树上”，想二分深度
- 11:43 “这一题甚至能不能把每一个给算出来？不能以 N 方算出来”
- 11:45 “我给每一层求个最大值” ✓✓
- 11:48 “插入一个串，这条路径上全加一，如果这是最大值，上面的每一层都要取一遍最大值” ✓✓
- 11:49 “字典树如果退化成链……难办了。现在我能高速算出来每一个，但这复杂度差得远了去了” ← 卡在这
- 12:45 “你不挨个算，你怎么能做到的呢？”
- 赛后你说的做法：“每插入一个串，把它顶上的全维护一遍，全部重新算改动的东西”——这就是维护 $mx[d]$ ，是对的

你已经有的：设 $mx[d]$ = 深度 d 上经过次数最多的那个结点的次数。那么 $f(i, j)$ = 最大的 d ，使得 $mx[d] \geq j$ 。这就是你 11:45 说的“每层最大值”。

你卡住的地方：“不把每个 f 都求出来，怎么能算出最后的值？”

先说清楚一件事：每个 f 都求出来了，都存在数组 $f[1..i]$ 里，只是不重新求。

你的算法：每插入一个串，把 $j = 1..i$ 的 f 全部重算一遍，再加一遍 xor 。一共 $1 + 2 + \dots + n \approx n^2/2$ 次， $n = 5 \times 10^5$ 时是 1.25×10^{11} 。

正解：插入第 i 个串以后， f 数组里只有很少几格会变。只改变了的格子，改的时候把和也跟着改：

- 先把这格原来的 $f \text{ xor } j$ 从和里减掉
- 改 f
- 再把新的 $f \text{ xor } j$ 加回去

没变的格子一格都不碰，它们对和的贡献原样留着。这就是“不挨个算也知道和”的全部秘密：和是一直维护着的，不是每次重新加出来的。

哪几格会变？插入一个串，它经过的每一层只有一个结点 $+1$ ，所以每层的 $mx[d]$ 最多 $+1$ 。 $mx[d]$ 从 m 变成 $m + 1$ 时：

- 对 $j \leq m$ ：本来就有 $mx[d] \geq j$ ，没变
- 对 $j \geq m + 2$ ：还是 $mx[d] < j$ ，没变
- 只有 $j = m + 1$ ：条件从“不成立”变成“成立”，所以只有 $f(m+1)$ 可能变大，变成 $\max(f(m+1), d)$

所以插入第 i 个串，最多改 $|s_i| + 1$ 格。所有串加起来 $\sum |s_i| + n \leq 1.5 \times 10^6$ 次。

用样例 1 走一遍 (a、ab、abc、abcd，答案 0、6、4、12)：

插入	新加的一项	哪一层的 mx 变了 → 改哪一格 f	f 数组 ($f_1 f_2 \dots$)	和
a	$j=1: 0 \text{ xor } 1 = 1$, 和 = 1	第 1 层 $mx\ 0 \rightarrow 1 \rightarrow f_1: 0 \rightarrow 1$, 和 $-1 + 0$	1	0
ab	$j=2: 0 \text{ xor } 2 = 2$, 和 = 2	第 1 层 $mx\ 1 \rightarrow 2 \rightarrow f_2: 0 \rightarrow 1$, 和 $-2 + 3$; 第 2 层 $mx\ 0 \rightarrow 1 \rightarrow f_1: 1 \rightarrow 2$, 和 $-0 + 3$	2 1	6
abc	$j=3: +3$, 和 = 9	$f_3: 0 \rightarrow 1$ ($-3 + 2$); $f_2: 1 \rightarrow 2$ ($-3 + 0$); $f_1: 2 \rightarrow 3$ ($-3 + 2$)	3 2 1	4
abcd	$j=4: +4$, 和 = 8	$f_4: 0 \rightarrow 1$ ($-4 + 5$); $f_3: 1 \rightarrow 2$ ($-2 + 1$); $f_2: 2 \rightarrow 3$ ($-0 + 1$); $f_1: 3 \rightarrow 4$ ($-2 + 5$)	4 3 2 1	12

每一行都只动了“这个串经过的层数”那么多格。动画讲义里有样例 2 的逐步动画（8 个串、有分叉、有重复串）。

一共改 $\sum |s_i|$ 次, $O(\sum |s_i|)$ 。核心就这几行（完整代码 正解代码/B_毛线.cpp）：

```
ans += (0 ^ i);
for (int d = 0; d <= len; d++) {
    // u 走到第 d 层的结点
    cnt[u]++;
    if (cnt[u] > mx[d]) {
        mx[d] = cnt[u];
        int j = mx[d];
        if (d > f[j]) {
            ans -= (f[j] ^ j);
            f[j] = d;
            ans += (f[j] ^ j);
        }
    }
}
```

大样例 5 个全过，最大的 0.25 秒。

稳拿的 10 分：子任务 1 ($n \leq 10, |s| \leq 50$) 按定义暴力，正解代码/暴力/B_暴力.cpp 30 行。你 14:01 说“性价比不高”——10 分钟 10 分，比 D 那 5 分钟写错的 solve2 性价比高得多。

下次识别信号：

- “对每个前缀 i 都要输出答案” → 想增量：加一个元素，哪些量会变、变多少
- “ $f(j)$ = 最大的 d 使得 $g(d) \geq j$ ” → f 和 g 互为“转置”，g 的一次 +1 只改一个 f

专项：字典树模板题练手（洛谷 P8306）；然后自己把 B 从空白写一遍。

C (30 分, $O(n^2)$)

题目本质：设 X = 前缀的（红 - 黄）， Y = 前缀的（红 - 蓝）。段 (j, i) 是好的 $\Leftrightarrow X[j] \leq X[i]$ 且 $Y[j] \leq Y[i]$ 。

DP： $f[i] = \max(f[i-1], \max \text{ 好的 } j \text{ 的 } f[j] + i - j)$ 。

你的思路复原：

- 11:55 “要知道上一次切割点” → 11:58 写出 $O(n^2)$ DP ✓

- 12:01 “只要知道 $A - B$ 的值”“ $A - C$ ” ✓ (就是前缀差 X 、 Y)
- 12:04 “如果我以…… i 减去 $f[i]$ ” ✓ (正解里的关键量 $u = i - f[i]$)
- 12:05 “30 分手到擒来”，但 13:36 才开始写
- 13:40 第一版写成了严格大于，样例 1 输出 1 → 13:48 “大于等于我搞错了” ✓

做对的：DP 定义、前缀差、 $i - f[i]$ 这个量都想到了；样例抓住了“大于等于”。

做错的：12:05 想到 30 分却推迟了 1.5 小时才写。按执行卡“先交暴力”，12:05 就该花 10 分钟交上。

为什么 2×10^5 和 2×10^6 有区别 (你 12:05 问的)： 2×10^5 允许 $O(n \log^2 n)$ (二维偏序，CDQ 分治)，可以拿 60 分； 2×10^6 在 1 秒内只能 $O(n)$ 或很轻的 $O(n \log n)$ ，必须找题目的特殊结构。

正解思路 (供了解，现阶段拿 30 分就是对的)：

1. 设 $u[i] = i - f[i]$ ，意思是“前 i 格里最少有几格不在好段里”。转移： $u[i] = \min(u[i-1] + 1, \text{所有 } X[j] \leq X[i] \text{ 且 } Y[j] \leq Y[i] \text{ 的 } j \text{ 的 } u[j])$
2. 特殊结构：有红子的格子， X 、 Y 都不减；没有红子的格子， X 、 Y 都不增、且至少一个减。点 (X, Y) 每一步要么往右上走、要么往左下走，两维永远同方向
3. 往右上走时，新“看得到”的点只在一列和一行上 → 每列、每行各维护一个有序表
4. 往左下走时， u 只可能不变或 $+1$ → 只要判断“有没有 u 值等于当前 u 的点在左下方” → 每个 u 值维护一条阶梯 (极小点集合)
5. 只有“没红子而且没被覆盖”的格子才需要记下来

$O(n \log n)$ ， 2×10^6 各种分布最慢约 0.4 秒。和 $O(n^2)$ 暴力随机对拍 900 组全对。代码 正解代码/C_棋子.cpp。

下次识别信号：“段内 $A \geq B$ 且 $A \geq C$ ” → 前缀差 + 二维偏序； n 到百万级 → 一定有特殊结构，先看每一步能怎么变。

专项 (有余力再做)：一维版本“段和 ≥ 0 ，总长度最长”用树状数组做；二维偏序入门洛谷 P3810。

D 🐱 (0 分，TLE 然后 CE)

题目本质：村民排成一条链，相邻两个村民之间最多夹一个狼。左看的村民定死了“前面有几个狼”，右看的村民定死了“后面有几个狼”。

你的思路复原：

- 12:08 看 checker ✓ (执行卡第四节的规则，这次做到了)
- 12:52 子任务 2：“第一个人肯定说 0，不是 0 直接拖出去；比它多 1 的话左边那个是狼” ✓ 方向对
- 12:57 “列出所有情况，总共 8 种……太复杂了，只能拿个十分”
- 13:53 暴力直接搬 checker 的 `isValid` ✓ 聪明
- 14:04 子任务 2 写成“ $b[i-1] > b[i]$ 就输出 1” ✗

14:01 那版暴力为什么 0 分：

```
int t = i;
wolves.clear();
while (t != 0) {
    wolves.push_back(t % 2);
    t /= 2;
}
```

`wolves` 的长度是 `i` 的二进制位数，不是 `n`。`isValid` 第一句就检查长度，长度不够直接 `false`。结果只有最高位是 1（第 `n` 个人是狼）的分布被试过，“全是村民”（`i = 0`）也从没试过。

- 题面样例三组答案：001、-1、10101——恰好最后一个都是狼或者无解，所以样例过了
- 大样例 `sample2` 就是子任务 1 的 1000 组数据，一跑就错 277 组。你下载了，没跑

改法：for (int k = 0; k < n; k++) `wolves.push_back (i >> k & 1);`。改完就是 10 分。

14:09 那版为什么 CE：solve 里调用了 `solve2`，但 `solve2` 写在 `solve` 下面，没声明。桌面上 T4 可执行文件是 14:00 编译的，`T4.cpp` 是 14:09 改的——交之前没编译。而且就算编译过了，`if (n > 20)` `solve2 (n);` 后面没有 `return`，还会接着跑 2^n 暴力。

子任务 2（全是 L，15 分）怎么做：左看的村民 `i` 要求“前面恰好 `bi` 个狼”。从左往右，每个人两种状态：

- `i` 是村民：前面的狼数必须等于 `bi`
- `i` 是狼：`i-1` 必须是村民，前面的狼数 = `b(i-1) + 1`

每个状态的狼数都是确定的， $O(n)$ DP。你 12:52 说的“第一个人肯定说 0”就是这个 DP 的第一步。

正解（供了解）：

1. 如果知道狼的总数 `W`，右看的村民也能换成“前面有 `W - bi` 个狼”，就是上面那个两状态 DP， $O(n)$
2. 看相邻两个村民 `i → j` (`j = i+1` 或 `i+2`)：
 - 都左看：`b[j] = b[i] + 中间狼数`，和 `W` 无关
 - 都右看：`b[j] = b[i] - 中间狼数`，和 `W` 无关
 - 一左一右：只有一个 `W` 能走（左→右：`W = b[i] + b[j] + 中间狼数`；右→左：`W = b[i] + b[j] - 中间狼数`）
3. 所以一共只有 $O(n)$ 条“挑 `W` 的边”。把每个位置“能不能从前一个、前两个村民走过来”写成 2×2 的 0/1 矩阵，线段树维护乘积；枚举 `W` 时只把这个 `W` 的边打开、看能不能走到终点、再关上。 $O(n \log n)$
4. 找到 `W` 后普通 DP 回溯出方案；`n = 1` 时一个人当狼总是合法

大样例全部用 checker 规则校验合法；和穷举暴力随机对拍 10500 组（包括判 -1）全对； $\sum n = 2 \times 10^6$ 最慢 0.43 秒。代码 正解代码/D_狼人.cpp。

下次识别信号：有一个“全局未知量”（这里是总狼数）→ 看它在哪些地方被钉死；如果只在少数地方被钉死，就枚举它、每次只改那几处。

五、你赛时的七个“我觉得”，逐个判

你说的	判	理由
“10 的 6 次方，说明有通用的方法，而不是进行暴力” (A, 10:16)	✗	暴力不只是骗分：它是给你小反例、替你算答案的工具
“记录所有出现过的位置太多了” (A, 10:59)	这个判断对	你想的那种状态确实太多；换成“第几位 + 进位”只有 2n 个
“不会是编译器的问题吧” (A, 11:28)	差不多对	不是编译器，是 Code::Blocks 启动程序时没处理文件夹名里的括号（第十一节）
“现在我能高速算出来每一个，但这复杂度差得远” (B, 11:49)	估算对，结论错	每次全重算确实是 $O(n^2)$ ；缺的是“只改变了的那几格”。退化成链不是问题：插入总代价 = $\sum s $
“ 2×10^5 和 2×10^6 有啥区别” (C, 12:05)	有区别	前者允许 $O(n \log^2 n)$ ，后者必须找结构
“我认为我的思路没有问题” (A, 13:07)	✗，但你下一步做对了	反例 11011。你 10 分钟后去写暴力，这正是执行卡“我觉得我是对的 → 跑数据证明”的做法
“第二题十分，性价比不高” (14:04)	✗	B 暴力 10 分钟 10 分；你去写的 D 子任务 2 思路不对，还 CE

六、暴力为什么来晚了（你说的三条，录音里都有）

你说的原因	录音里的证据	以后怎么办
① 我知道代码错了，但不知道为什么错。我需要的是一个大数据，不是大数据	10:48 大样例 $947 \neq 671$ 、12:28 $729 \neq 671$ ：大样例只告诉你“错了”。13:02 你手编 10、110、1110，全对——手编的数据太整齐，碰不到反例	小反例让机器去找：暴力 + gen 从 $n = 1$ 开始 + 对拍。13:22 对拍错的第一组就是反例，13:24 你一手算就明白了
② 我能合理解释，认为代码非常正确	12:42 “我这个算法就是这个样子，为什么不对呢？”；13:07 “但我认为我的思路没有问题”	这时候再怎么说服自己都没用，只有反例能说服你。执行卡第四节：“我是对的”→ 1 分钟跑一个数据证明
③ 一开始那个等式就是暴力，但当时没认出来	10:16 “10 的 6 次方……而不是进行暴力”；11:16 写下 $a + b - (0 + b) = a$ ；12:13 “找一个很强的样例，关键我自己也算不出来答案”；13:17 “有可能是你生成两个”	认暴力的办法：式子长成“答案 = 在所有 X 里取最小 / 最大”，X 能一个个列出来，for 循环就是暴力

三条合起来是一件事：你把暴力当成“数据小的时候骗分用的”。所以 10:16 看到 10^6 就不写了。可是这场暴力真正的价值在另外两处：给你一个小反例，替你算出你算不出的答案。这两件事和数据范围无关， $n = 10^6$ 的题照样用得上。

你 13:17 → 13:35 的 18 分钟证明了你 would 用这个办法。下次要做的只是把它提前到“第一次觉得不对劲”的时候：10:48（大样例不对）或 12:13（算不出答案）。

七、和 Day1 比：执行卡做到了哪些

规则	这场	证据
必须 freopen	✓	A、C 都拿分了
工具 5 分钟修不好就回终端	✓	12:31 Code::Blocks 抽风，直接改用终端；环境损失从 30 分钟降到 12 分钟
先看 checker	✓	12:08 读了 D 的 checker，后来暴力直接搬了 <code>isValid</code>
“我觉得我是对的”→ 跑数据证明	✓（晚了一点）	13:07 说完，13:17 写暴力、13:22 找到反例
用暴力对拍	✓（晚了 2 小时）	对拍一开，2 分钟就找到反例
T+0:15 四题都读完	✗	B 11:39、C 11:52、D 12:07 才读（T+1:57）
T+1:30 至少两题暴力交上	✗	第一次提交 T+2:35
每题有预算，到点换题	✗	A 用了 65%
先交暴力，再想正解	✗	C 的 30 分 12:05 想到、13:51 才交；B 的 10 分没写
最后 30 分钟：编译无警告、大样例再跑一遍	✗	D 没跑大样例、最后一交没编译

结论：环境类、工具类的规则基本做到了；时间类、先拿分类的规则一条都没做到。下场重点是后者。

八、止损点

时刻	信号	应该做	实际
10:48	大样例不对，而且不知道为什么	停，先写暴力（哪怕是 BFS 搜最少项数）	继续空想 50 分钟
11:07	写出了 $a = P - N$	5 分钟写“枚举 N”的暴力，打表	13:17 才写
T+1:00 (11:10)	还没交过任何题	停下来，挑一道写暴力交上	继续想 A
12:05	自己说“30 分手到擒来”	立刻写 C 的 $O(n^2)$ ，10 分钟	推迟到 13:36
12:28	输出 $729 \neq 671$	不交，去写暴力	12:45、13:06 各交一次错的
14:04	离结束 6 分钟，D 的新想法没把握	不再改 D；或者只修暴力的枚举	写了 solve2，没编译就交

九、经验教训

错因档案里的复发

#	病因	这是第几次	这场的表现
1	读题漏条件	2	C 写成“严格大于”（样例抓住了，没丢分）
3	不止损、碰运气	2	A 已知 $729 \neq 671$ ，改系数试 853，还交了两次错的
4	不按子任务拿分	2	A 子任务 1、B 子任务 1、D 子任务 1/2 的暴力都没及时写
5	时间分配失控	2	A 占 65%（Day1 是 55%）
8	环境 / 工具问题	2	约 12 分钟（Day1 约 30 分钟）。其中 10 分钟是文件夹名 <code>problem_185(1)</code> 的括号，见第十一节

不计复发的两条：#2 复杂度盲区——B 你估的 $O(n^2)$ 是对的，缺的是增量的想法，算知识缺口；#6 找理由代替验证——13:07 你说“我认为我的思路没有问题”，但 10 分钟后就去写暴力验证了，这正是规则要你做的。

新病因

#	病因	这场的表现
9	交的不是测过的那份	A 在 5 个地方有 5 份；C 写进了 <code>T2.cpp</code> 、B 写进了 <code>T3.cpp</code> ；12:45 交了 12:22 的旧文件（RE）；13:06 交了已知过不了样例的；14:09 交了没编译的（CE）
10	只拿小样例验证	D 暴力过了题面样例就交，大样例（1000 组）没跑；A 对拍时 <code>gen</code> 还是空模板，“1000 组全对”是假的

题型规则（写进执行卡）

1. 二进制 + 加减 + 最少项数 → 数位 DP，状态记进位
2. 写出等式 → 枚举一边就是暴力
3. 估复杂度按“总和”：字典树插入总代价 = $\sum |s|$
4. 对每个前缀都要答案 → 增量维护，只改变了的项
5. 枚举 n 位的 0/1 分布：`mask >> k & 1`， k 从 0 到 $n-1$

调试规则

1. 对拍前 `cat 1.txt` 看一眼输入不是空的
2. 题面样例过了不算，大样例全跑一遍
3. 已知过不了样例的代码不交

比赛策略

1. $T+1:00$ 还没交过任何题 → 立刻停，挑一道写暴力交上
2. 自己说出“这 X 分手到擒来” → 立刻去拿，不许推迟

3. 最后 10 分钟：只交刚编译过、跑过大样例的代码

十、行动清单

现在补（按顺序，约 3 小时）：

专项	练习	预计
数位 DP 带进位	不看代码重写 A 的数位 DP；做 LeetCode 2571	40 分钟
字典树 + 增量维护	洛谷 P8306 练手；然后从空白写 B	1 小时
小状态 DP	写 D 子任务 2（全是 L）的两状态 DP，用 D 的 checker 验证	40 分钟
写暴力	四题的暴力各默写一遍（正解代码/暴力/），每题限时 10 分钟	40 分钟

下次比赛：按新版《赛前执行卡》执行，重点是“三、时间检查点”和“五、交之前”两节。桌面换成新版 bag（tools/vm/bag_v2/），它会帮你做“测的就是交的”和“交之前检查”。

赛后复盘时带上：归档 ZIP + AI 上下文 TXT、录音转写、虚拟机桌面导出。这次题目列表页快照里就有你的提交记录，以后归档报“本人提交 0”也不用慌。

十一、环境两个坑（你问的）

1. Syntax error: word unexpected（11:22 和 12:31，一共约 10 分钟）

不是你代码的问题，是文件夹名里的括号。

- 10:18 你“下载第二遍，防止附件有用”，第二份自动叫 problem_185(1).zip，解压出来的文件夹名带括号
- Code::Blocks 20.03 在 Linux 上运行程序时，路径只把空格转义（源码 compilergcc.cpp 里的 EscapeSpaces），括号不管；再交给 cb_console_runner，它用 system() 调 /bin/sh 去执行（源码 ConsoleRunner/main.cpp）
- sh 把路径里的（当成语法。我在这边复现了：把 ../problem_185(1)/T1 交给 sh，输出一字不差是 Syntax error: word unexpected (expecting ")")；同一个程序放进不带括号的文件夹，正常输出 474
- 所以你当时的几个观察都对得上：“别的代码没问题”（别的文件夹没括号）、“注释掉还不对”（和代码无关）、11:30 在终端直接运行就好了、12:31 在桌面那个 problem_185(1) 里“又抽风了”

怎么办：附件只下一遍；已经带括号就右键重命名，把（1）去掉；或者直接用终端运行。bag v2 的 run.sh 给路径加了引号，不受影响。

2. 复制粘贴失灵（10:44、12:38）

你发现的规律是对的：复制的那个窗口关了，就粘不了。

- Linux 图形界面（X11）里，Ctrl+C 并没有把内容存进一个公共的剪贴板，只是那个窗口声明“我手里有东西”。粘贴时，系统去问那个窗口要。窗口关了，就没人给了

- Windows 会自己存一份，所以你以前没遇到过
- 另外两个容易混的：在终端里粘贴是 Ctrl+Shift+V (Ctrl+V 不行)，复制是 Ctrl+Shift+C (Ctrl+C 是中断程序)；鼠标选中文字、按滚轮也能粘贴，这是另一套“选区”，同样要求原窗口还开着

怎么办：复制完先别关原来的窗口；要搬整份代码，用文件（cp、另存为），别靠剪贴板。

附一：正解代码与验证记录

题	文件	做法	验证
A	正解代码/A_乘法.cpp	数位 DP, $O(n)$	样例 2/2；12 位以内所有 a 加 13~16 位随机，共 5295 个，和暴力一致；你最后 AC 的代码也全部一致
B	正解代码/B_毛线.cpp	字典树 + mx[d] 增量， $O(\sum s)$	样例 5/5（最大 0.25 秒）；随机 500 组和按定义暴力一致
C	正解代码/C_棋子.cpp	见四.C, $O(n \log n)$	样例 3/3 + 题面样例；随机 900 组 ($n \leq 5000$, 7 种分布) 和 $O(n^2)$ 一致； 2×10^6 最慢约 0.4 秒
D	正解代码/D_狼人.cpp	见四.D, $O(n \log n)$	大样例全部按 checker 规则校验合法；随机 10500 组和穷举一致； $\sum n = 2 \times 10^6$ 最慢 0.43 秒
暴力	正解代码/暴力/	四题各一份，分别对应能拿的子任务分	都和正解对过

附二：赛时代码（赛时代码/，文件名里是北京时间）

文件	说明
A_v0_数1的个数_11-23.cpp	打表：数样例里 1 的个数（474）
A_v1_只试最高位借位_约10-45_抄自paper.cpp	从 paper.txt 抄出来的第一版，大样例 947
A_v2_连续1段_12-22_推断为12-45提交RE.cpp	越界，本地也会段错误
A_v3_改系数_12-33.cpp	大样例 853
A_v4_13-02_13-06提交WA.cpp	大样例 729，反例 11011
A_暴力_枚举b_13-22.cpp、A_造数据_13-23.cpp	你的暴力和 gen
A_v5_加check_13-33_13-35提交AC.cpp	100 分
C_v1_写在T2里_严格大于_13-40.cpp	样例 1 输出 1
C_v2_写在debug里_13-49_13-51提交30分.cpp	30 分
D_v1_暴力_约14-00_14-01提交0分_还原.cpp	由 14:09 的源码去掉 solve2 还原
D_v2_加solve2_14-09_14-09提交CE.cpp	CE
B_写在T3里_未完成_14-06.cpp	开头

附三：给归档脚本的两个修正建议

1. 题名解析：D 被记成“0 Compile Error”，是把题目列表里的状态单元格拼进了题名。只取标题链接里的文字就行

2. 本人提交 403 时的兜底：比赛题目列表页本身就有“递交 状态 题目 递交者 时间 内存 语言 递交时间”表，和每题最后得分。403 时从这张表里解析，就不会报“完全未提交”

附四：我是怎么做出来的（你问的）

先说实话：四题我都没见过。我的做法和你 13:17 以后做的是同一套：先写暴力，再猜，每个猜想拿几千组随机数据和暴力对，错了就扔。区别只是我从一开始就这么做，而且我跑一千组只要几秒。C 和 D 我都猜错过好几次，是暴力告诉我错了。

题	我怎么想的	猜错过几次
A	认出来了：“用最少几个 $\pm 2^k$ 凑出一个数”是有名的问题（有符号二进制 / NAF，LeetCode 2571 就是它）。二进制加减最少项的标准做法是从低位做、记进位，就写了数位 DP。然后用 $a + b - b$ 的暴力把 12 位以内全部对了一遍	0
B	推出来的，没靠猜：① 选 j 个串的最长公共前缀 = 字典树里“下面至少有 j 个串”的最深结点；② 每层取最大值得到 $mx[d]$ ， $f(j)$ 就是“ $mx[d] \geq j$ 的最大 d ”；③ 问自己“插入一个串，到底有几个量变了”，发现每层 mx 最多 +1，只动一格 f ；④ 用样例 1 手算一遍（就是四.B 那张表）再写代码	0
C	最难。先写 $O(n^2)$ 暴力，再把条件写成前缀差 X 、 Y （和你 12:01 想的一样）。标准做法 CDQ 分治是 $O(n \log^2 n)$ ， 2×10^6 在 1 秒里跑不完，只能找结构。猜错了 4 次，见下表。最后发现“每一格 X 、 Y 同涨同跌”，才找到对的做法：5000 组 0 错，再上 C++，900 组 + 大样例 + 2×10^6 计时	4
D	① 先想“如果知道总狼数 W ，就是简单 DP”；② 发现 W 只在“左看的人和右看的人挨着”的地方被钉死；③ 第一版带着“所有可能的 W ”往后推，结果对了，但我用程序专门去找坏数据，找到集合能涨到 14 个，最坏会变成 $O(n^2)$ ；④ 换成“枚举 W ，每次只打开这个 W 的几条边”，用线段树维护 2×2 的 0/1 矩阵；⑤ 第一版最坏 1.5 秒，超时，改成非递归线段树 + 剪枝，降到 0.43 秒	1 (慢)

C 题我猜错的四次（每次都是拿暴力对拍发现的）：

猜想	结果
转移只需要看“同一行或同一列”的点	4000 组里错 1 组（最小反例 13 位）
只看同一行、同一列上最近的那个点	4000 组里错 89 组
只看最近一个能转移过来的点	4000 组里错 1672 组
$u = i - f[i]$ 只由点的位置决定	3000 组里 117 处反例
有红子的格子往右上走、没红子的往左下走，分开处理	5000 组 0 错 ✓

第一条猜想 4000 组才错 1 组。如果我只跑 100 组，就会以为它是对的。所以对拍要跑到几千组，而且数据要小（ $n \leq 15$ ），小数据才容易撞上反例。